Software engineering final assignment

Name: Tushar Pirwani

Student ID: 22464293

Practical Date/Time: Tuesday November 19, 2024 09:00 – 09:30

Introduction

This assignment involves the development of a program that reads a maze from a text file and improves the display of maze structures. The task required creating a program that converts a given maze, represented as a string, into a more readable format using box-drawing characters. The project was focused on improving the visual clarity of the maze, which is represented with simple ASCII characters, by replacing certain characters with more structured box-drawing characters, such as `┌`, `┐`, `└`, and `┘`, for walls and boundaries.

The problem revolved around reading a maze from a file and converting its various components into box-drawing characters. This was done to enhance the readability of the maze for the user, as traditional ASCII characters such as `+`, `-`, and `|` can be hard to distinguish when trying to view or analyze the structure. The task required mapping these characters to their respective box-drawing equivalents, while ensuring that the maze's structure and layout remained intact and clear.

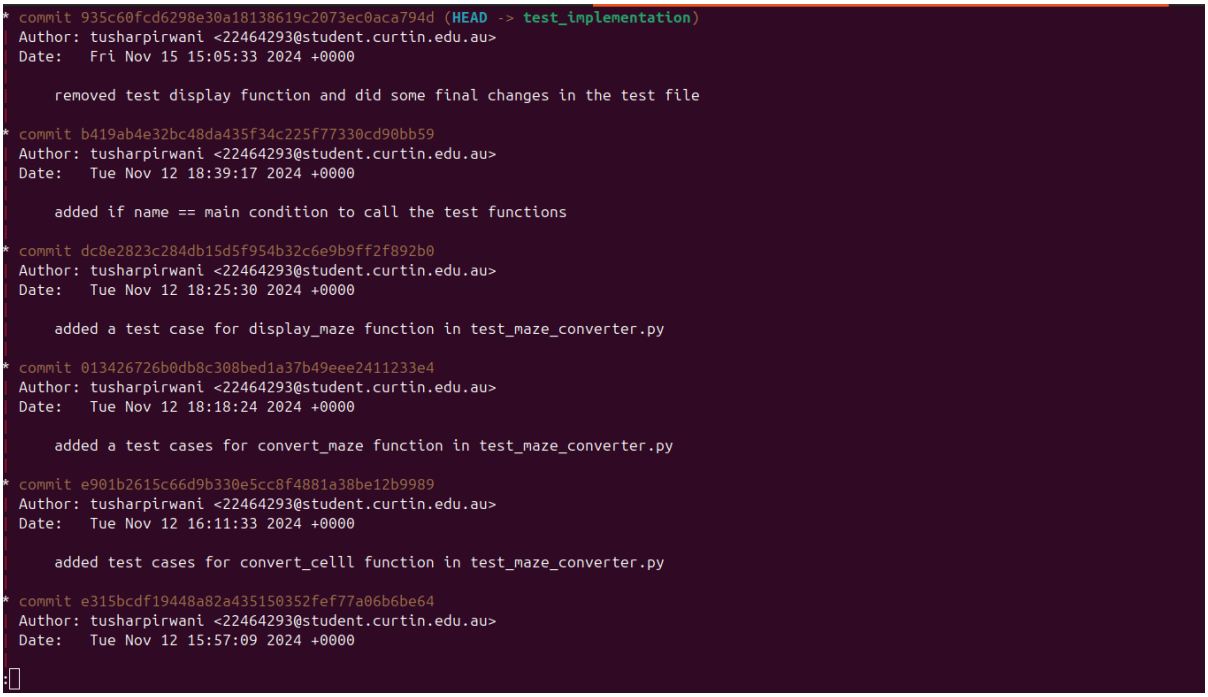
The project was developed using a modular approach, where each core functionality was implemented in separate functions, making the code easier to manage, test, and maintain. Version control was implemented using Git, with multiple branches for different features, such as the modularity_implementation and test_implementation branches. Testing was performed by creating a dedicated test_maze_converter.py file, which included various test cases to ensure that the maze conversion logic was working correctly. Both white-box and black-box approaches of testing were used in the testing process.

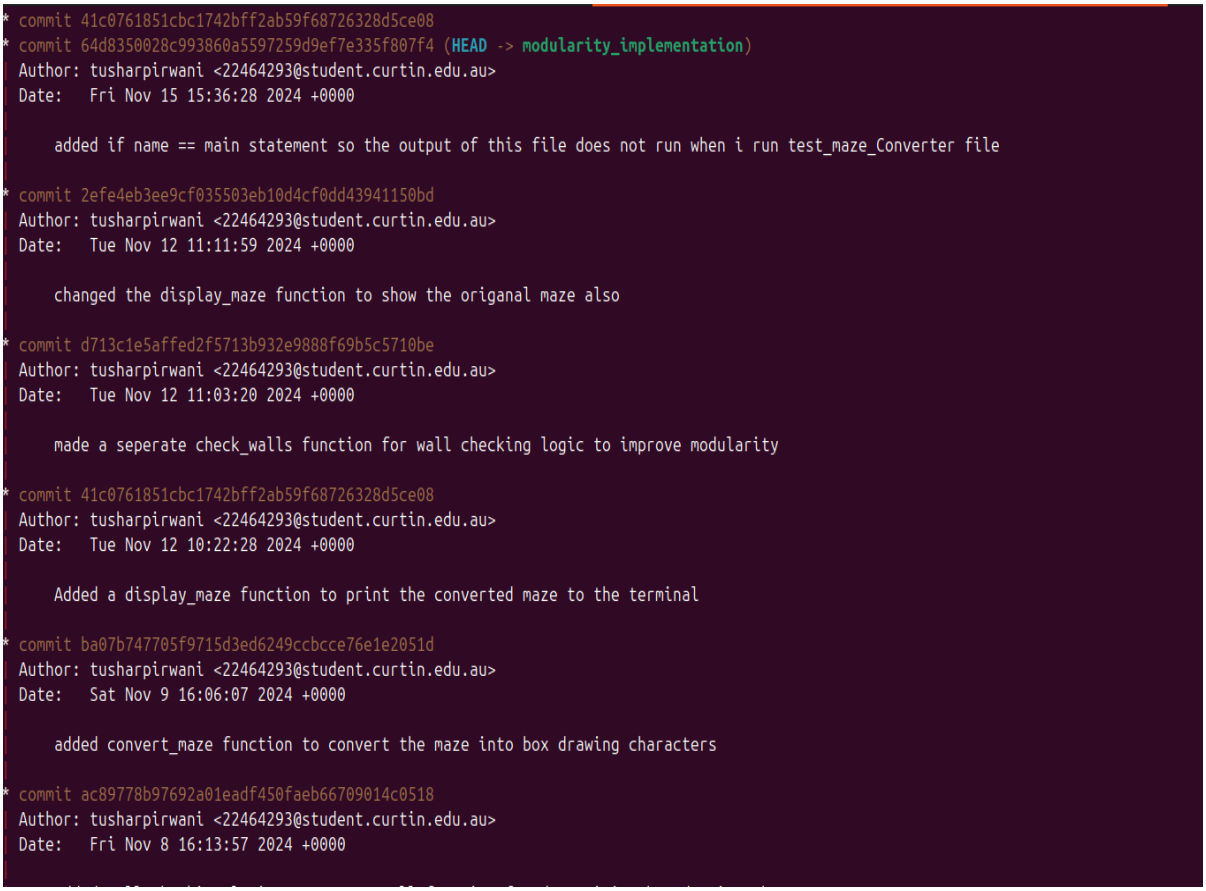
Version control

Version control for this project was managed using Git. The project was structured across multiple branches to separate different aspects of development:

1. **`modularity_implementation` branch**: This branch contained the core implementation of functions in the `maze_converter.py` file which gave the desired output of a box character maze compared with the original maze.
2. **`test_implementation` branch**: This branch was dedicated to testing the program's functionality using the `test_maze_converter.py` file. It included various test cases for different modules to check their logic and functionality
3. **`master` branch**: This branch served as the main branch for this project. Once the development and testing branches were completed, their changes in the form of commits, were merged into the master branch, creating a final and stable version of the project.

Throughout the project, commits were made at logical intervals to document progress, such as after implementing core functions, writing new test cases. Commits were made after implementation of every function in the file like check_walls or convert_cell function.This ensured traceability and made it easy to review the project's history. Below is the evidence of the commits of both the branches.





Modularity design and Implementation

Module Descriptions

1. **`read_maze`**
    - **Purpose and Responsibilities**:
      Reads a maze from a text file and returns it as a list of strings, where each string represents a line in the maze.
    - **Imports**: The filename as a string.
    - **Exports**: A list of strings representing the maze lines.
2. **`check_walls`**
    - **Purpose and Responsibilities**:
      Checks the neighboring cells of a specific position (x, y) in the maze for walls (`+`, `-`, or `|`) and returns their status as a tuple (left, right, up, down).
    - **Imports**: The maze lines (list of strings) and coordinates (x, y).
    - **Exports**: A tuple indicating the presence of walls in the four directions.
3. **`convert_cell`**
    - **Purpose and Responsibilities**:
      Determines the appropriate box-drawing character for a specific cell based on its neighbors' wall status.
    - **Imports**: The maze lines (list of strings) and coordinates (x, y).
    - **Exports**: A single character representing the cell.
4. **`convert_maze`**
    - **Purpose and Responsibilities**:
      Iterates through the maze, processes each cell, and converts it into a new maze using box-drawing characters.
    - **Imports**: The maze lines (list of strings).
    - **Exports**: A list of strings representing the converted maze.
5. **`display_maze`**
    - **Purpose and Responsibilities**:
      Prints both the original and converted maze for visual comparison.
    - **Imports**: The original and converted mazes (lists of strings).
    - **Exports**: Printed output to the terminal.

Checklist Review

Using the modularity checklist from lecture/worksheet 7, the following points were addressed:

- **Cohesion**: Each function performs a single, clearly defined task. For example, read_maze is solely responsible for file reading, while convert_cell only determines the character for a single cell.
- **Coupling**: Dependencies between functions are minimized. Each function communicates through well-defined inputs and outputs without tightly linking to others.
- **Reusability**: Functions like check_walls and convert_cell can be reused in other projects requiring similar maze-processing logic.
- **Testability**: The modular design makes it easy to test individual functions independently, ensuring correctness at every stage.

Iterative Design Changes/ Refactoring

1. **Separation of Wall Checking**:
   Initially, I added the wall-checking logic in the convert_cell function but later I created a different function for it, check_walls. This improved the overall modularity of my code as now each function handled their own specific task with minimal dependency. I also mentioned these changes in my commits.

2. **Streamlined File Handling**:
   Initially, I added file reading logic in the main function but later I created a new function for this as well called read_maze.

3. **Enhanced Display**:
   Initially, my display function only displayed the converted maze but later I made sure it also displays the original maze for comparison between both of them.

Test design and Implementation

Testing Methodology:

For testing the maze converter functions, I did not use any formal testing frameworks. Instead, I imported the necessary modules from the maze_converter.py file to test individual functions. This approach allowed me to directly call the functions and verify their outputs manually. I tested the following functions:

- **read_maze(filename)**: Ensured that the maze is correctly read from a file and returned as a list of strings.
- **check_walls(maze_lines, x, y)**: Verified that the function correctly identifies neighboring walls.
- **convert_cell(maze_lines, x, y)**: Checked if the correct box-drawing character is applied based on neighboring walls.
- **convert_maze(maze_lines)**: Validated that the entire maze is converted into box-drawing characters.

Black-box testing:

Test case	Module	Purpose	Inputs	Expected outputs
TC1	read_maze	Verify file reading.	File test_maze.txt with a simple 3 line maze "+---+, | |, +-+-+"	"+---+, | |, +-+-+"
TC2	convert_maze	Verify entire maze conversion.	"+---+, | |, +---+"	“ ┌─────┐, │ │, └─────┘ ”

White-box testing:

Test case	Module	Purpose	Inputs	Expected outputs
TC1	check_walls	Validate top-left corner handling.	Maze lines, top-left corner`(0, 0)`.	Tuple: (False, True, False, True)
TC2	check_walls	Validate top-right corner handling.	Maze lines, top-right corner `(4, 0)`	Tuple: (True, False, False, True)
TC3	check_walls	Validate bottom-left corner	Maze lines, bottom-left	Tuple: (False, True, True, False)

		handling.	corner`(0, 2)`	
TC4	check_walls	Validate bottom-right corner handling.	Maze lines, bottom-right corner`(4, 2)`	Tuple: (True, False, True, False)
TC4	convert_cell	Validate vertical wall replacement.	Maze lines, cell `(4, 1)`.	Box-drawing character: '`│`'
TC6	convert_cell	Validate top-left corner replacement.	Maze lines, cell `(0, 0)`.	Box-drawing character: '`┌`'
TC7	convert_cell	Validate bottom-left corner replacement.	Maze lines, cell `(0, 2)`.	Box-drawing character: '`└`'
TC8	convert_cell	Validate top-right corner replacement.	Maze lines, cell `(4, 2)`.	Box-drawing character: '`┐`'

Summary of work

Module	Completion Status	Test Design Completion	Test Implementation Completion	Test Success Status
read_maze(filename)	Yes	Yes	Yes	Yes
check_walls(maze_lines, x, y)	Yes	Yes	Yes	Yes
convert_cell(maze_lines, x, y)	Yes	Yes	Yes	Yes
convert_maze(maze_lines)	Yes	Yes	Yes	Yes

Explanation of Table:

- **Module Completion**: All core modules were completed successfully.
- **Test Design Completion**: Test design was completed for all modules.
- **Test Implementation Completion**: Tests for each function were manually implemented.

- **Test Success Status**: All tests passed, though some edge cases (such as malformed files and, boundary conditions) were identified during testing and have been addressed through code improvements.

Discussion

Reflection:

The maze converter project allowed me to successfully implement functions for reading, converting, and displaying the maze. I learned valuable lessons in handling data transformations and formatting output. A key achievement was implementing the maze conversion to box-drawing characters. However, testing manually without a formal framework posed challenges, and I had to address issues like boundary conditions and malformed files. These challenges helped me improve my debugging skills and reinforced the importance of thorough testing.

Limitations and Future Improvements:

- **Error Handling**: More robust error handling for malformed files and inconsistent row lengths could be added.
- **Boundary Handling**: The check_walls function could be improved to handle maze boundaries more effectively.
- **Display Alignment**: The display_maze function could be enhanced to handle irregular row lengths better.
- **Testing Framework**: Switching to a formal testing framework like unittest would streamline testing and improve test coverage.
- **Performance**: Larger mazes may require optimizations for performance.
- **Extended Features**: Future improvements could include features like automatic maze solving or graphical visualizations.

References

1. **ASCII Art and Box-Drawing Characters** – Wikipedia. (n.d.). Retrieved from https://en.wikipedia.org/wiki/Box-drawing_character